PatchPilot 🩹 — 给每一次代码变更一颗「会解释、能验证、守门禁」的审查大脑

第四届开放原子大赛 · 2026 开源行业解决方案创新赛 · 赛道一「开源基础软件与解决方案」作品介绍文档

项目名称	PatchPilot（补丁飞行员）— Agentic Code Reviewer
参赛赛道	赛道一 · 开源基础软件与解决方案（AI 开发工具链 / 智能体框架）
团队	个人参赛（项目负责人：支一郎）
代码仓库	`https://github.com/Zhiyilang/patchpilot`
国内镜像	`https://gitee.com/ZhiYIlang/patchpilot`（Gitee，国内可直接访问，内容同步）
开源协议	MIT
演示视频	见提交表单视频链接（61.2 秒，真实运行实录）
技术栈	Python 3.10+ · 零第三方运行时依赖 · Nebius Token Factory（OpenAI 兼容）+ NVIDIA Nemotron

91/91	**12 条**	**3 类**	**0**
离线自测断言全部通过	确定性静态规则	CI 全绿 (3.10/3.11/3.12)	运行时第三方依赖

一、项目背景：代码审查是软件工程里最后一块「人力飞地」

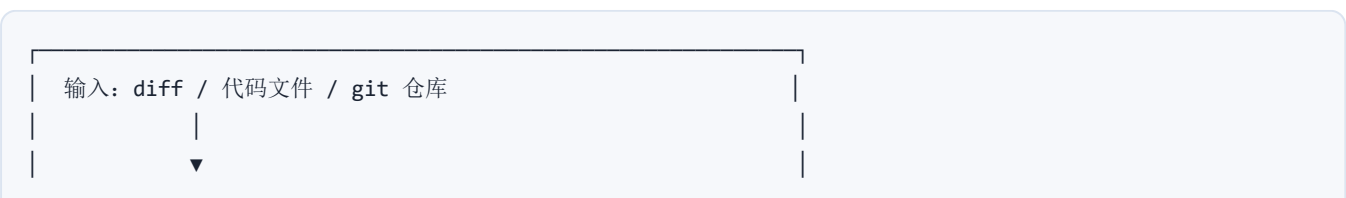
现代工程的每个环节都已被工具链自动化，唯独**代码审查**仍高度依赖人的时间与经验：PR 排队等 review、审查者被重复性风格问题消耗、安全问题在「看一眼」中漏过，而 RPA/自动化脚本开发者这一群体尤甚——大量低代码平台产出的流程脚本缺乏系统化审查手段。

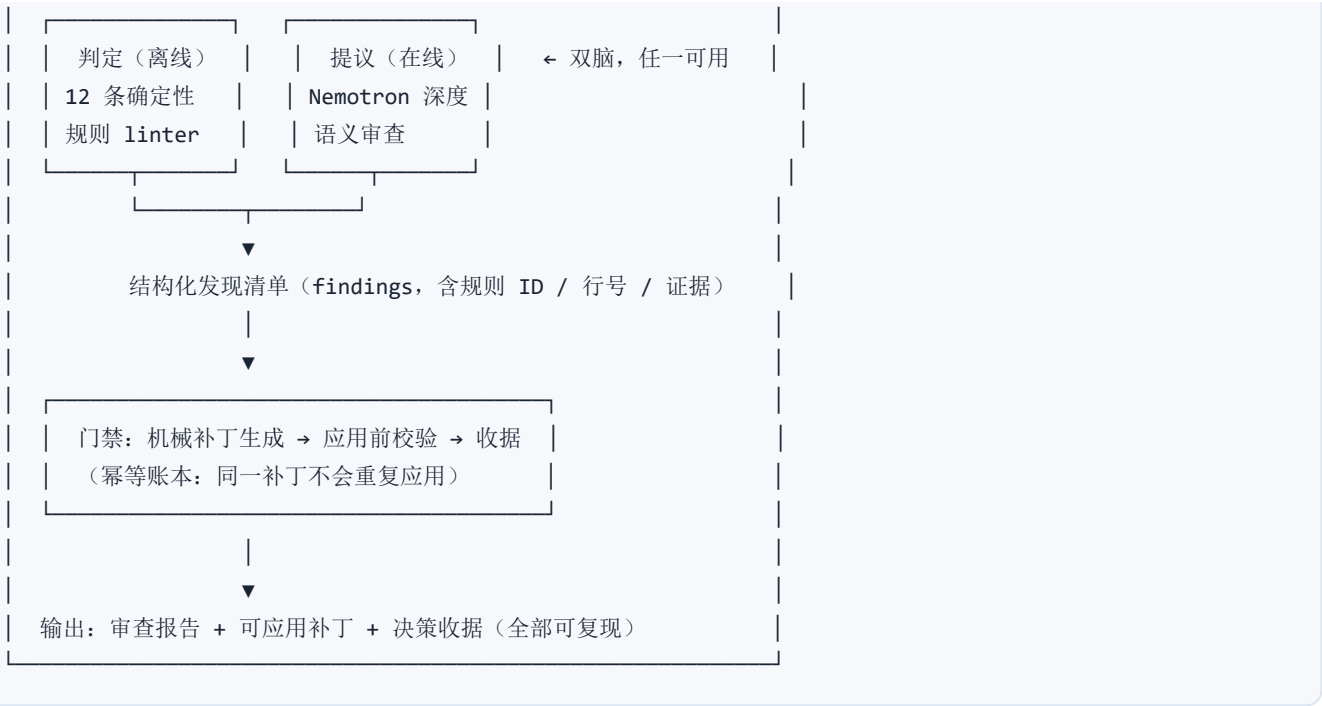
与此同时，直接接入大模型做 review 又面临三个工程现实：**(1)** 模型输出不稳定，同样的代码两次审查结论可能不同；**(2)** 无 API Key 的离线环境（内网、涉密、学生机房）直接被排除在外；**(3)** 模型「建议」与「可执行补丁」之间存在断层，改不改、谁来验，没有闭环。

PatchPilot 要解决的就是这个问题：**一个任何环境下都能跑、结论可复现、建议可执行、风险有门禁的代码审查智能体。**项目源于作者一年企业级 RPA（影刀/UiBot）开发中积累的真实审查痛点，而非虚构场景。

二、技术架构：「提议 / 判定 / 门禁」三分范式 + 双脑设计

PatchPilot 的核心设计决策是把「智能」与「可靠」拆到两条独立通道，让确定性部分永远可复现，让概率性部分永远被约束：





关键设计决策

1. **判定与提议分离。**离线 linter（12 条规则：10 条正则模式 + 2 项结构性检查）永远可用且输出确定；Nemotron 深度审查只负责「离线规则够不着」的语义层发现。无 API Key 时系统**完全可用**（离线模式），而非降级成摆设。
2. **补丁必须过门禁。**LLM 提议的每个修复在应用前都要通过确定性校验，且记入幂等账本——同一补丁对同一目标不会二次应用，杜绝「AI 越改越乱」。
3. **结论必须带证据。**每条发现都绑定规则 ID、行号与命中证据，评审与复盘都能回溯到具体代码位置，而非接受一句「模型觉得有问题」。
4. **零运行时依赖。**纯 Python 标准库实现核心链路，任何装有 Python 3.10+ 的机器开箱即跑——这是对「内网/离线/教育场景」的硬承诺。

三、应用场景

- **企业 RPA / 低代码流程脚本治理**：影刀、UiBot 等平台产出的流程脚本缺少系统化审查手段，PatchPilot 可作为 CI 门禁接入（作者本职场景，已有一线使用反馈）。
- **离线/内网研发团队**：无外网 API 的环境下，离线 linter + 机械补丁仍提供完整的「发现问题—给出修复—应用留痕」闭环。
- **AI 工程教学与示范**：双脑分离 + 门禁范式是讲解「如何约束 LLM 输出」的活教材；91 条离线断言本身即是一套可执行的规格说明。
- **个人开发者的 PR 自检**：提交前本地跑一遍，把风格与常见缺陷拦截在推送之前。

四、创新点

1. **「提议/判定/门禁」三分范式**：把智能体的生成能力与系统的可靠性拆到两条通道——概率部分只提议，确定部分做判定与放行。相比「让 LLM 一条龙包办」的主流做法，本项目的审查结论可复现、补丁应用可审计。
2. **双脑 + 优雅降级**：离线确定性 linter 与在线语义审查互为备份，任一通道缺失系统仍完整可用，而非简单报错退出。

3. **幂等补丁账本**：把「AI 修改代码」这一高风险动作变成可重放、可验证的账本操作，是智能体安全落地的关键工程件。
4. **规格即可执行**：91 条离线断言既是测试套件也是行为规格，`python eval.py` 一条命令向任何人自证系统状态。

五、评审维度对照

技术创新（30%）

「提议/判定/门禁」三分范式针对现有 LLM review 工具的三个未解决问题（输出不可复现、离线不可用、建议与执行断层）给出了系统性方案，非现成工具的简单组合；双脑分离与幂等账本为自主代码修改提供了可验证的安全边界。

场景落地（30%）

项目源自作者一年企业级 RPA 开发的一线审查痛点，面向真实可部署场景（CI 门禁 / 内网环境 / 教学示范）。验证数据来自真实运行：`python eval.py` → **91 passed, 0 failed**（离线、无 Key、零依赖）；Web 控制台冒烟测试 HTTP 200；CI 在三个 Python 版本上全绿。演示视频为真实运行实录而非编排表演。

开源治理（20%）

MIT 协议；README 中英双语；架构设计文档（`DESIGN.md`）公开；CI 矩阵覆盖 Python 3.10/3.11/3.12，任何贡献不破坏既有规格；91 条断言即行为规格，新人上手路径 = clone → `python eval.py` → 全绿即环境就绪。仓库提交历史与发布节奏全部公开可查。

长期发展（20%）

项目是作者「AI Agent 工程」开源主线（已公开 20+ 个仓库、全部 CI 全绿，覆盖代码审查 / 合同风控 / 收据解析 / 量化裁决等场景）的核心节点，本仓库沿用了同一套经多项目验证的工程规范（零依赖、断言即规格、双语文档）。路线图明确：离线规则持续扩充、更多语言与低代码平台适配、与 CI 平台的原生集成；同一「三分范式」已在作者多个生产向项目中复用验证。

六、复现指南（三分钟）

```
git clone https://github.com/Zhiyilang/patchpilot
cd patchpilot
python eval.py                  # → 91 passed, 0 failed（离线自证）
python -m patchpilot.cli examples/bad_code.py   # 离线 linter 完整可用
# 可选：export NEBIUS_API_KEY=<key> 启用 Nemotron 深度审查
# 国内网络可用 Gitee 镜像：git clone https://gitee.com/ZhiYIlang/patchpilot
```

环境要求：Python 3.10+，无第三方依赖。完整架构说明见仓库 `DESIGN.md`。

七、结语

PatchPilot 不追求「最聪明的审查器」，而追求**任何环境下都敢接进生产流水线的审查器**：确定性部分永远可复现，概率性部分永远被门禁约束，每一次修改都在账本里留痕。这既是软件工程对智能体的合理要求，也是本项

目对「开源赋能行业创新，方案驱动场景落地」这一主题的具体回答。
